Secure Backscatter IoT: Authenticated FSK Backscatter Communication for Battery-Free Smart Locks

ECSE 4660/6660 Internetworking of Things

4/28/2026

Andrew Napieraj, napieraj@rpi.edu, ECSE 4660

Andrew Keefe, keefea3@rpi.edu, ECSE 4660

1. Objective

This project develops a secure, battery-free smart-lock communication link based on RF backscatter and demonstrates that a passive tag can transmit authenticated commands to a fixed receiver while consuming only microwatts of switching energy at the tag itself. The tag does not generate any RF carrier; it modulates the reflection coefficient of an exciter signal supplied by separate infrastructure. The research question is: can we build a backscatter link that is robust enough to recover short authenticated payloads at meter-scale ranges using only off-the-shelf SDR and microcontroller hardware, while cryptographically defending against replay, tampering, and forgery attacks?

The system targets battery-free smart-lock and access-control scenarios. A user actuates a magnetic reed switch on the tag to send an "OPEN" command; the receiver authenticates the packet using AES-CTR encryption and an AES-CMAC tag, and rejects any packet whose monotonic counter has been seen before. The same architecture generalizes to any low-power binary command channel where the tag can be exclusively reflective (no oscillator) and the receiver provides authentication and replay protection.

Beyond the demo itself, the project is a small but complete case study in what backscatter physical-layer choices (modulation, sub-carrier frequency selection, synchronization tracking) cost in receiver complexity and what they buy in robustness — a tradeoff that determines whether ambient backscatter is practical for real-world IoT deployment.

2. Innovation and Literature Review

Backscatter communication has been studied extensively. RFID systems have used continuous-wave (CW) excitation for decades, and recent work such as HitchHike (SenSys '16) and FreeRider (CoNEXT '17) has demonstrated WiFi-compatible backscatter using existing access points as exciters. Two papers in particular informed the receiver design here:

- Dunna et al., SyncScatter (NSDI '21): identifies symbol-level synchronization as the dominant driver of receiver sensitivity in backscatter and introduces a hierarchical wake-up + sync stage that keeps average tag power around 30 μW while achieving 30+ m range. This work motivated our use of frequency-domain decision metrics rather than envelope thresholds.
- Satori (2025): emphasizes channel-distortion handling and CFO tracking in WiFi backscatter, which informed the receiver's two-stage CFO correction loop.

What is novel about this project compared to that prior work is the combination of three properties in a single, reproducible build:

- Authenticated payloads at the physical layer. Most academic backscatter work focuses on raw bit recovery and explicitly defers security to higher layers. We integrate AES-CTR

(NIST SP 800-38A) and AES-CMAC (NIST SP 800-38B / RFC 4493) directly into the tag firmware, so every transmitted packet is encrypted, authenticated, and bound to a monotonic counter that survives power cycles.

- Non-harmonic binary FSK on the sub-carrier. We pick 1.0 kHz for '1' bits and 1.7 kHz for '0' bits specifically because neither is a harmonic of the other (1 kHz square wave: 1, 3, 5, 7, 9 kHz; 1.7 kHz square wave: 1.7, 5.1, 8.5 kHz — disjoint). This eliminates the cross-contamination that would corrupt a more naive choice like 1 kHz / 2 kHz, where the second harmonic of '1' lands directly on the '0' detector.
- An entirely off-the-shelf hardware path. Exciter is a Pluto SDR driven by a Raspberry Pi over USB; the tag is an MSP430G2553 LaunchPad driving an Analog Devices EVAL-ADG902EBZ RF switch eval board; the receiver is a second Pluto SDR. No custom RF integrated circuits, no transmission lines, no impedance-tuning networks — making this directly reproducible in a course lab setting.

Existing reproductions of HitchHike and similar systems either require custom ASICs (SyncScatter), dedicated tone generators (LoRa-Backscatter), or full WiFi-frame manipulation. None of those are accessible to a typical IoT-class student team in a one-semester project. Our system trades range and throughput for simplicity and includes the security layer those works defer.

3. Methodology

We build a one-way authenticated backscatter link with three physical components: an exciter that radiates a continuous 2.48 GHz CW tone, a tag that modulates the reflection coefficient of that tone with an AES-encrypted payload, and a receiver that demodulates and authenticates the resulting backscattered packet. Figure 1 shows the system block diagram.

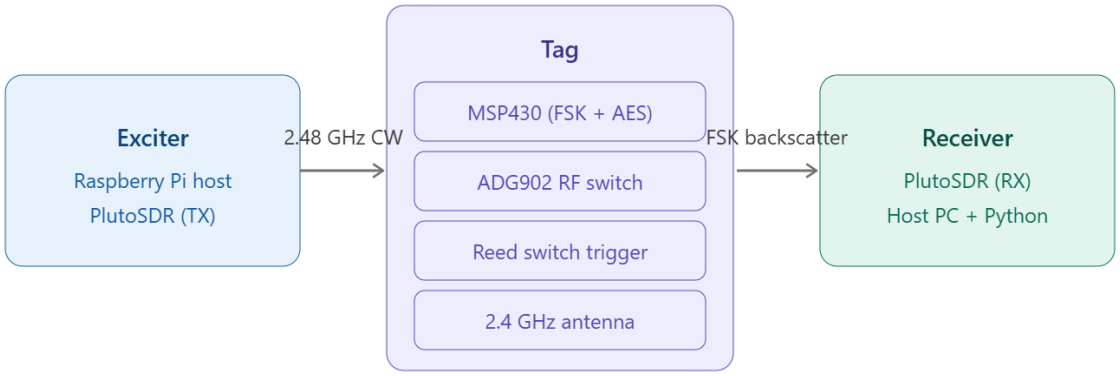

Figure 1. System block diagram of the secure backscatter link.

3.1 Physical Layer: FSK Sub-carrier Modulation

The MSP430 generates a square-wave sub-carrier on Timer_A0, output to GPIO P1.1, which drives the CTRL pin of the ADG902 RF switch through a 10 kΩ series resistor. Switching the ADG902 between its two reflection states modulates the backscattered signal. To encode bit values, we change the sub-carrier frequency rather than gating it on/off (OOK):

- Bit '1': sub-carrier at 1.0 kHz (TA0CCR0 = 499, 100 ticks per bit, 50 ms wall-clock).

- Bit '0': sub-carrier at 1.7 kHz (TA0CCR0 = 293, 170 ticks per bit, ~50 ms wall-clock).

FSK is preferred over OOK for two reasons. First, the receiver's bit decision becomes self-normalizing: it compares the energy at 1.0 kHz against the energy at 1.7 kHz, rather than thresholding absolute power. There is no adaptive threshold to drift in the presence of a long string of identical bits. Second, the tag is always transmitting energy regardless of the bit value, which means the receiver can continuously track the carrier and perform CFO correction even during long runs of zeros.

3.2 Cryptographic Layer

Each on-air packet has the following layout, transmitted MSB-first per byte:

- Preamble: 0xAA (1 byte) — clock recovery and bit-sync pattern (alternating 1s and 0s).
- Sync: 0x7E (1 byte) — packet start delimiter.
- Counter: 4 bytes, big-endian — a monotonic counter persisted to MSP430 flash that increments at every transmission.
- Ciphertext: AES-CTR(payload), where the payload is the fixed command "OPEN" (4 bytes) and the IV is constructed as counter || 12 zero bytes (NIST SP 800-38A).
- Authentication tag: first 8 bytes of AES-CMAC(counter || ciphertext), per NIST SP 800-38B / RFC 4493.

The receiver verifies in this order: (1) decode preamble and sync, (2) recompute AES-CMAC over the received counter and ciphertext and compare to the received tag, (3) reject any packet whose counter is less than or equal to the highest counter observed so far, (4) decrypt with AES-CTR and check that the plaintext is "OPEN". Failure at any stage rejects the packet without unlocking. The receiver persists its highest-seen counter to disk so replay attacks cannot succeed across restarts.

The reed switch on the tag gates RF transmission entirely. With a magnet present, the sub-carrier output is disabled and the tag radiates nothing; removing a magnet enables the sub-carrier and triggers a packet transmission. This means an attacker observing the link can only ever see legitimate transmissions, never solicit them.

3.3 Receiver DSP Pipeline

The receiver runs as a Python program on a host PC with a PlutoSDR connected over USB. Per RX buffer (65,536 complex samples at 1 Msps), the pipeline performs:

- DC blocking (first-order IIR, $\alpha = 0.9998$, cutoff ≈ 32 Hz) to remove the LO leakage that otherwise dominates the spectrum near baseband.
- Carrier-tracking via FFT peak finding with sticky lock-in: the strongest in-band peak is selected provided it is at least 10 dB above the median noise; otherwise the previous lock is held.
- Two-stage residual-CFO correction (Satori-inspired): a slow EMA estimates the average residual frequency offset; a faster EMA tracks short-term fluctuations around it.
- Per-chip dual-frequency Goertzel-style detection: for each 50 ms chip, compute m_f1 (energy at 1.0 kHz) and m_f0 (energy at 1.7 kHz) by mixing the AM envelope with reference exponentials and taking the magnitude of the resulting complex coefficient.
- Differential chip decision: the chip is '1' if (m_f1 − m_f0) > 0, '0' otherwise. A small dead-zone rejects pure-noise chips; a presence floor rejects buffers where the carrier itself has dropped out.

- Multi-phase chip slicing: Four parallel candidate phase offsets are tracked simultaneously to handle the case where the buffer boundary does not align with a chip boundary.
- Header search: the resulting bit stream is scanned for the 16-bit preamble+sync pattern (0xAA 0x7E) with a Hamming-distance tolerance, and the trailing payload is passed to the security layer for verification.

4. Hardware/Software Used

Table 1 summarizes the hardware and software stack used in the build.

Component / Tool	Role in System	Notes
Raspberry Pi (host)	Drives the exciter Pluto SDR; runs pluto_exciter.py as a systemd service.	Powered from wall supply for always-on operation.
Pluto SDR (#1, exciter)	Generates a continuous-wave 2.48 GHz tone as the backscatter excitation signal.	Configurable TX gain; cyclic IQ buffer.
MSP430G2553 LaunchPad	Tag MCU. Drives sub-carrier and runs the security firmware.	Pins used: P1.0 (red LED), P1.1 (Timer_A0 sub-carrier output to ADG902 CTRL via 10 kΩ), P1.6 (green LED for '1' bits), P1.3 (reed switch input), GND.
EVAL-ADG902EBZ	Single-pole RF switch eval board acting as the reflection-modulation element of the tag.	Single-antenna mode: J1=RF1=antenna; J6=CTRL (driven by P1.1); J7=VDD (2.6 V external); J3=GND.
2.4 GHz antenna (tag)	Captures exciter energy and re-radiates the modulated reflection.	Connected to ADG902 J1 via short SMA jumper.
Reed switch	User-actuated trigger. Normally-open reed wired to P1.3 with internal pull-up; closes (grounds the pin) when a magnet is brought near, which gates the sub-carrier OFF. Removing the magnet opens the contact, drives P1.3 high, and triggers transmission.	Wired to MSP430 GPIO with internal pull resistor; gates the sub-carrier output.
Pluto SDR (#2, receiver)	Captures the backscattered signal at 2.48 GHz, 1 Msps.	RX gain 45 dB, manual; 1 MHz analog bandwidth.
Host PC (receiver)	Runs receiver/main.py — the FSK demodulator, packet decoder, and security verifier.	Live GUI: baseband, carrier-detail, FSK metrics windows.
MSP430 firmware (C)	BackscatterTag/Msp_FSK_secure.c — Timer_A0 PWM, AES-CTR, AES-CMAC, monotonic counter persistence to flash.	Code Composer Studio / msp430-gcc.
Receiver software (Python)	receiver/ package — DSP, decoder, secure_packet.py for crypto verification.	Python 3 + numpy + scipy + matplotlib + pyadi-iio (libiio).

Component / Tool	Role in System	Notes
Cryptography	AES-CTR (NIST SP 800-38A) + AES-CMAC (NIST SP 800-38B / RFC 4493).	128-bit key shared between tag and receiver out-of-band.

All software and firmware is published in the project repository at https://github.com/AndrewKeefe918/Secure-Backscatter-IoT, under the MIT license.

5. Final Results and Demo

We evaluated the system on a benchtop setup in a typical office environment, with the exciter, tag, and receiver clustered within roughly one meter of each other. The exciter Pluto SDR was set to maximum on-board TX gain consistent with linear operation; the receiver Pluto SDR ran with manual RX gain at 45 dB and 1 Msps sample rate. Tests were run with continuous excitation, magnet present (tag transmitting), and magnet absent (tag idle).

5.1 Spectrum Verification

With the tag transmitting, the carrier-detail spectrum shows a clean carrier near 0 Hz baseband offset and clearly visible sidebands at ±1.0 kHz and ±1.7 kHz, corresponding to the two FSK sub-carrier frequencies. The peak at ±1.7 kHz tends to run a few dB stronger than the ±1.0 kHz peak, consistent with the message being slightly biased toward zero bits time-on-air. Measured sideband SNR is approximately +13.5 dB at the 1.0 kHz markers, though below the 20 dB ideal target. With the tag idle (no magnet), both sideband pairs disappear into the noise floor, confirming that the observed peaks are tag-generated rather than environmental interference.

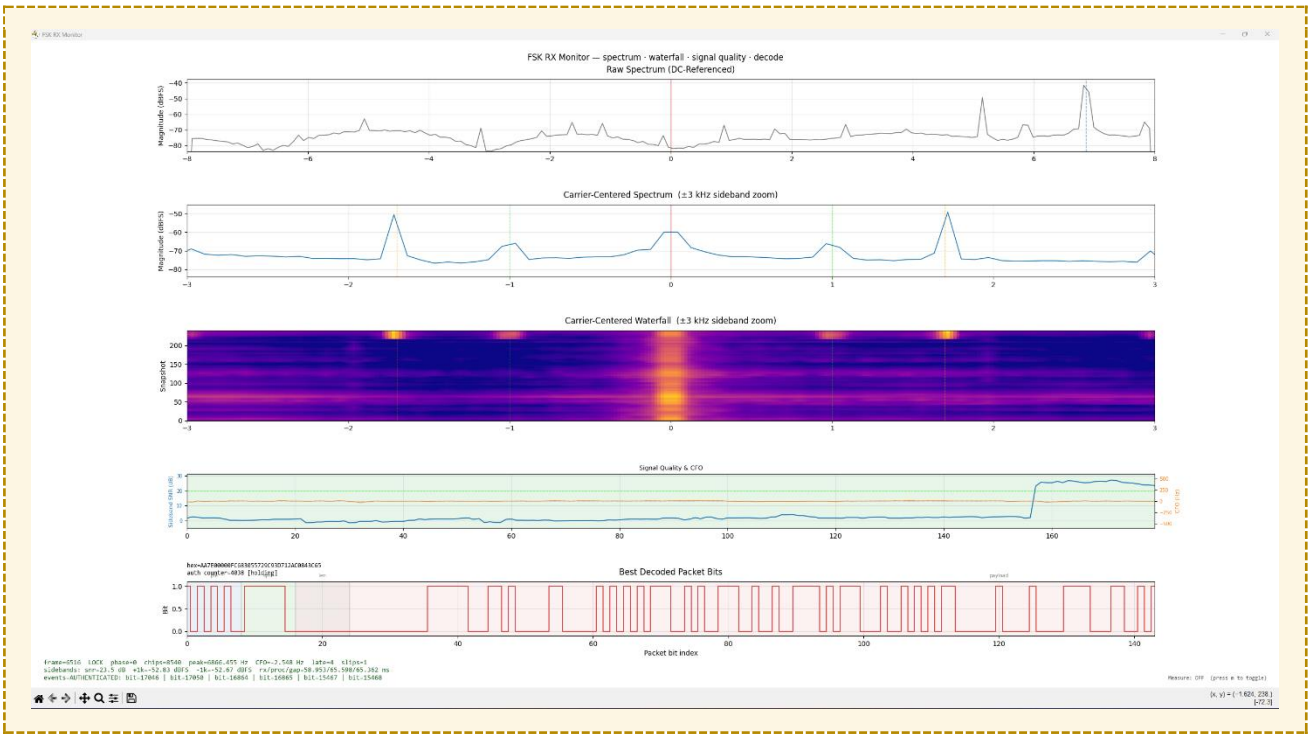

Figure 2. Live rx_monitor view during transmission, showing raw spectrum, carrier-centered spectrum, waterfall, signal quality, and recovered chip bits.

5.2 Demodulator Behavior

The receiver's FSK demodulator panel plots m_f1 (1.0 kHz energy) and m_f0 (1.7 kHz energy) per chip alongside the resulting binary decisions. During preamble (0xAA → 10101010) the two metric traces alternate cleanly, and during the body of the packet they track the encrypted ciphertext bits as expected. The differential decision rule produces stable chip outputs even when the absolute envelope amplitude varies, illustrating the SNR advantage of FSK over OOK on this channel.

```
PS C:\Users\andre\GitHub\Secure-Backscatter-IoT> python -m receiver.main
[RX STATUS] frame=5610 lock=0 phase=0 peak=+6820.7Hz cfo=+0.7Hz snr=-0.6dB proc=64.8ms late=2 slip=1 bits=0111111101111110101111111110111111111111110011111 decode=No live decode yet
[RX STATUS] frame=5625 lock=0 phase=0 peak=+6820.7Hz cfo=+11.2Hz snr=-0.6dB proc=64.9ms late=2 slip=1 bits=1111110111111111111110001111111111111101110101011011 decode=No live decode yet
[RX STATUS] frame=5640 lock=0 phase=0 peak=+6820.7Hz cfo=+6.7Hz snr=-1.3dB proc=65.8ms late=2 slip=1 bits=0111111111111101110101101101101001111101111111111 decode=No live decode yet
[RX STATUS] frame=5655 lock=0 phase=0 peak=+6820.7Hz cfo=-18.7Hz snr=-0.5dB proc=64.9ms late=2 slip=1 bits=1101011101101100111110111111111111111100101100001 decode=No live decode yet
[RX STATUS] frame=5670 lock=0 phase=0 peak=+6820.7Hz cfo=-1.4Hz snr=-0.9dB proc=64.8ms late=2 slip=1 bits=0111111111111111001010000011000100011110011111111 decode=No live decode yet
[RX STATUS] frame=5685 lock=0 phase=0 peak=+6820.7Hz cfo=-0.5Hz snr=-0.7dB proc=64.8ms late=2 slip=1 bits=1000011000100011110011111111110111000100111111111 decode=No live decode yet
[RX STATUS] frame=5700 lock=0 phase=0 peak=+6820.7Hz cfo=-0.5Hz snr=+0.2dB proc=65.7ms late=2 slip=1 bits=1111111101110001001111111101111110001100111101100 decode=No live decode yet
[RX STATUS] frame=5715 lock=0 phase=0 peak=+6835.9Hz cfo=-5.7Hz snr=+1.2dB proc=64.9ms late=2 slip=1 bits=1001111111111011111000110011101100011011110101010 decode=No live decode yet
[RX STATUS] frame=5730 lock=0 phase=0 peak=+6835.9Hz cfo=-2.3Hz snr=+1.2dB proc=64.6ms late=2 slip=1 bits=1100111011000110111101010101010101011111011101111 decode=No live decode yet
[RX STATUS] frame=5745 lock=0 phase=0 peak=+6835.9Hz cfo=+0.6Hz snr=+1.1dB proc=64.6ms late=2 slip=1 bits=1010101010101111101111011111111101011101111111111100 decode=No live decode yet
[RX STATUS] frame=5760 lock=0 phase=0 peak=+6835.9Hz cfo=+1.6Hz snr=+1.8dB proc=65.4ms late=2 slip=1 bits=1011111110101110111111111100011111100101011011 decode=No live decode yet
[RX STATUS] frame=5775 lock=0 phase=0 peak=+6835.9Hz cfo=-3.4Hz snr=+1.4dB proc=64.6ms late=2 slip=1 bits=1111111110001111110010101011011100111111111111 decode=No live decode yet
[RX STATUS] frame=5790 lock=0 phase=0 peak=+6835.9Hz cfo=-4.3Hz snr=-0.7dB proc=64.4ms late=2 slip=1 bits=0101011110111001111111111111010001010100001011011 decode=No live decode yet
[RX STATUS] frame=5805 lock=0 phase=0 peak=+6835.9Hz cfo=+2.6Hz snr=+0.8dB proc=64.6ms late=3 slip=1 bits=1111110100010101000101100111101110011101111101010 decode=No live decode yet
[RX STATUS] frame=5820 lock=0 phase=0 peak=+6835.9Hz cfo=+1.0Hz snr=+0.3dB proc=65.3ms late=3 slip=1 bits=1100111011100111101111110101010110100001110111110 decode=No live decode yet
[RX STATUS] frame=5835 lock=0 phase=0 peak=+6835.9Hz cfo=+0.3Hz snr=+0.2dB proc=64.3ms late=3 slip=1 bits=0111110101010110100011101111101011111100111101101 decode=No live decode yet
[RX STATUS] frame=5850 lock=0 phase=0 peak=+6835.9Hz cfo=+4.4Hz snr=+0.3dB proc=64.7ms late=3 slip=1 bits=1101111101011111100111010101011111111111111110101 decode=No live decode yet
[RX STATUS] frame=5865 lock=0 phase=0 peak=+6835.9Hz cfo=+7.5Hz snr=-0.1dB proc=64.7ms late=3 slip=1 bits=0101010111111111111111111010111110001011110101 decode=No live decode yet
[RX STATUS] frame=5880 lock=0 phase=0 peak=+6835.9Hz cfo=-0.5Hz snr=-0.0dB proc=65.6ms late=3 slip=1 bits=1101011111000101111101011110111111011111110100101 decode=No live decode yet
[RX STATUS] frame=5895 lock=0 phase=0 peak=+6835.9Hz cfo=+10.1Hz snr=-0.0dB proc=64.7ms late=3 slip=1 bits=1111101011110111111011111111110100101010110010101110 decode=No live decode yet
[RX STATUS] frame=5910 lock=0 phase=0 peak=+6835.9Hz cfo=+1.7Hz snr=-0.0dB proc=64.6ms late=3 slip=1 bits=1111101001010101011001011101110101110111000111001 decode=No live decode yet
[RX STATUS] frame=5925 lock=0 phase=0 peak=+6835.9Hz cfo=+2.4Hz snr=+0.4dB proc=64.7ms late=3 slip=1 bits=1011101110101110110001100111111101101110100101010 decode=No live decode yet
[RX STATUS] frame=5940 lock=0 phase=0 peak=+6835.9Hz cfo=+2.0Hz snr=+1.1dB proc=65.6ms late=3 slip=1 bits=1110011111110110111010010101001100011001110101010 decode=No live decode yet
[RX STATUS] frame=5955 lock=0 phase=0 peak=+6835.9Hz cfo=+10.2Hz snr=+1.1dB proc=64.7ms late=3 slip=1 bits=1110100101010011000110011110101000100011010111110 decode=No live decode yet
[RX STATUS] frame=5970 lock=0 phase=0 peak=+6835.9Hz cfo=+8.0Hz snr=+1.6dB proc=64.6ms late=3 slip=1 bits=0011110101000100011010111101111001010110010110100 decode=No live decode yet
[RX STATUS] frame=5985 lock=0 phase=0 peak=+6835.9Hz cfo=+9.4Hz snr=+1.7dB proc=64.6ms late=3 slip=1 bits=0111101111000101011001011000101100010100101101110110101 decode=No live decode yet
[RX STATUS] frame=6000 lock=0 phase=0 peak=+6835.9Hz cfo=+11.1Hz snr=+1.0dB proc=65.5ms late=3 slip=1 bits=1011000101001011011101101011101110111111000001011 decode=No live decode yet
[RX STATUS] frame=6015 lock=0 phase=0 peak=+6835.9Hz cfo=+10.2Hz snr=+1.3dB proc=64.9ms late=3 slip=1 bits=0111010101011101110111110000010111010011111111101 decode=No live decode yet
```

Figure 3. Receiver terminal output with the tag silent (reed switch closed). The receiver tracks the exciter peak at ~+6.8 kHz consistently, residual CFO stays within ±11 Hz, sideband SNR sits at −0.5 to +1.8 dB (below the +20 dB lock threshold), and the live-decode summary correctly reports "No live decode yet". The realtime budget is being met (proc ≈ 65 ms ≈ T_buf), with only minor jitter (late=2-3, slip=1).

5.3 End-to-End Packet Decode

When the magnet is held to the reed switch, the receiver consistently locks onto the preamble and decodes packets. A successful decode produces the following terminal output: header match with 0–2 bit errors, AES-CMAC verification PASS, monotonic counter check PASS, AES-CTR decryption yielding the plaintext "OPEN", and a final RX PACKET line indicating successful authenticated decode. Failures at any of these stages — corrupted bits beyond the tolerance, CMAC mismatch, replayed counter — are reported with the specific failure cause and do not unlock. [cached] means the same packet has already been verified on an earlier frame and is being re-detected from the receiver's rolling chip history; the live-decode summary refreshes every few frames (~15 RX buffers/sec at 65 ms/buffer), so the same packet typically re-appears as [cached] for several seconds after its first decode. A line without [cached] is the first-decode of that packet.

```
PS C:\Users\andre\GitHub\Secure-Backscatter-IoT> python -m receiver.main
[RX STATUS] frame=6345 lock=0 phase=0 peak=+6851.2Hz cfo=+5.3Hz snr=+2.7dB proc=64.9ms late=4 slip=1 bits=1011111111101101100111101011110101011111011010 decode=No live decode yet
[RX STATUS] frame=6360 lock=0 phase=0 peak=+6866.5Hz cfo=-9.6Hz snr=+2.8dB proc=65.5ms late=4 slip=1 bits=1010111110110101011111010101001101111101111110 decode=No live decode yet
[RX STATUS] frame=6375 lock=0 phase=0 peak=+6866.5Hz cfo=-4.4Hz snr=+1.7dB proc=64.9ms late=4 slip=1 bits=1111110010100100111011111101111110110101011110101 decode=No live decode yet
[RX STATUS] frame=6390 lock=1 phase=0 peak=+6866.5Hz cfo=-8.4Hz snr=+25.8dB proc=64.9ms late=4 slip=1 bits=1111001111110101010111110101111110111111110101010011111 decode=No live decode yet
[RX STATUS] frame=6405 lock=1 phase=0 peak=+6866.5Hz cfo=-14.4Hz snr=+23.8dB proc=64.8ms late=4 slip=1 bits=1101011111111101010101011111000000000000000000 decode=No live decode yet
[RX STATUS] frame=6420 lock=1 phase=0 peak=+6866.5Hz cfo=-8.1Hz snr=+26.7dB proc=65.6ms late=4 slip=1 bits=0111111000000000000000000000111111000111000011101 decode=No live decode yet
[RX STATUS] frame=6435 lock=1 phase=0 peak=+6866.5Hz cfo=+6.8Hz snr=+25.9dB proc=65.0ms late=4 slip=1 bits=0000000000001111110001110001111011011000100011101 decode=No live decode yet
[RX STATUS] frame=6450 lock=1 phase=0 peak=+6866.5Hz cfo=-3.2Hz snr=+26.2dB proc=64.9ms late=4 slip=1 bits=1110001110110110001000111011101110010011101111100 decode=No live decode yet
[RX STATUS] frame=6465 lock=1 phase=0 peak=+6866.5Hz cfo=+2.3Hz snr=+26.9dB proc=64.9ms late=4 slip=1 bits=0111011101110010011011011110011001110110011010001110 decode=No live decode yet
[RX STATUS] frame=6480 lock=1 phase=0 peak=+6866.5Hz cfo=+1.5Hz snr=+25.6dB proc=65.7ms late=4 slip=1 bits=1111001100111010011010001101000110111011110001101 decode=No live decode yet
[RX STATUS] frame=6495 lock=1 phase=0 peak=+6866.5Hz cfo=-2.0Hz snr=+25.0dB proc=65.0ms late=4 slip=1 bits=0110100011010001101110111110001010110000100000000 decode=phase=0 off=0 bit=105 pol=norm herr=0 weak=0 counter=4039 plai
ntext=b'OPEN'  AUTHENTICATED [cached]
[RX STATUS] frame=6510 lock=1 phase=0 peak=+6866.5Hz cfo=-1.6Hz snr=+23.8dB proc=64.8ms late=4 slip=1 bits=0111100010101100010000000000000000000000000000000 decode=No live decode yet
[RX STATUS] frame=6525 lock=1 phase=0 peak=+6866.5Hz cfo=+4.6Hz snr=+25.3dB proc=64.5ms late=4 slip=1 bits=0000000000000000000000000000000000000000101011100 decode=No live decode yet
[RX STATUS] frame=6540 lock=1 phase=0 peak=+6866.5Hz cfo=+8.0Hz snr=+26.5dB proc=65.5ms late=4 slip=1 bits=0000000000000000000101011001111110000000000000 decode=No live decode yet
[RX STATUS] frame=6555 lock=1 phase=0 peak=+6866.5Hz cfo=+1.5Hz snr=+26.0dB proc=64.7ms late=4 slip=1 bits=0001010111001111110000000000000000000000111111001 decode=No live decode yet
[RX STATUS] frame=6570 lock=1 phase=0 peak=+6866.5Hz cfo=+6.4Hz snr=+24.9dB proc=64.9ms late=4 slip=1 bits=0000000000000000001111110010000011101101001111 decode=No live decode yet
[RX STATUS] frame=6585 lock=1 phase=0 peak=+6866.5Hz cfo=+8.2Hz snr=+24.7dB proc=64.7ms late=4 slip=1 bits=1110010000001110110110011110000101000100110011101 decode=No live decode yet
[RX STATUS] frame=6600 lock=1 phase=0 peak=+6866.5Hz cfo=+7.3Hz snr=+25.7dB proc=65.1ms late=4 slip=1 bits=0011110000101000100100110110100011110000100011110010 decode=No live decode yet
[RX STATUS] frame=6615 lock=1 phase=0 peak=+6866.5Hz cfo=+7.3Hz snr=+26.4dB proc=64.7ms late=4 slip=1 bits=1001001110100011110001000111001001111011010100000 decode=No live decode yet
[RX STATUS] frame=6630 lock=1 phase=0 peak=+6866.5Hz cfo=+7.3Hz snr=+27.4dB proc=64.8ms late=4 slip=1 bits=0100011100100111110110100000110010011111100111 decode=phase=25000 off=0 bit=112 pol=norm herr=0 weak=0 counter=4040
plaintext=b'OPEN'  AUTHENTICATED [cached]
[RX STATUS] frame=6645 lock=1 phase=0 peak=+6881.7Hz cfo=+2.5Hz snr=+29.4dB proc=64.8ms late=4 slip=1 bits=0100000110010011111110011111111111111111111111 decode=No live decode yet
[RX STATUS] frame=6660 lock=1 phase=0 peak=+6881.7Hz cfo=-4.9Hz snr=+30.5dB proc=65.6ms late=4 slip=1 bits=1001111111111111111111111111111111111111111111 decode=No live decode yet
[RX STATUS] frame=6675 lock=1 phase=0 peak=+6881.7Hz cfo=-2.1Hz snr=+30.9dB proc=64.7ms late=4 slip=1 bits=1111111111111111111111111111111111101010100111111100 decode=No live decode yet
[RX STATUS] frame=6690 lock=1 phase=0 peak=+6881.7Hz cfo=-8.1Hz snr=+30.6dB proc=64.7ms late=4 slip=1 bits=1111111111101010100111110000000000000000000011 decode=No live decode yet
[RX STATUS] frame=6705 lock=1 phase=0 peak=+6881.7Hz cfo=-3.9Hz snr=+31.3dB proc=64.7ms late=4 slip=1 bits=1111000000000000000000000111111001001001100010101 decode=No live decode yet
[RX STATUS] frame=6720 lock=1 phase=0 peak=+6881.7Hz cfo=-1.0Hz snr=+29.8dB proc=65.6ms late=4 slip=1 bits=0001111110010010110001010100001100000000111110011 decode=No live decode yet
```

Figure 4. Terminal log of a successful authenticated packet decode.

5.4 Security Demonstrations

The receiver/demo_attacks.py script exercises the security layer offline against three attack scenarios:

- Legitimate traffic: a freshly generated packet with the next valid counter is accepted.
- Replay: re-submitting a previously accepted packet is rejected because its counter is no longer strictly greater than the receiver's highest-seen counter.
- Tampering: flipping one bit in either the ciphertext or the counter field causes the AES-CMAC tag to mismatch, and the packet is rejected before decryption is even attempted.
- Forgery: an attacker who lacks the AES key cannot construct a valid CMAC, so any forged packet is rejected with the same outcome as tampering.

These checks demonstrate that the link provides confidentiality, integrity, authenticity, and replay protection, the four properties expected of an authenticated-encryption channel.

```
Using key: 2b7e151628aed2a6abf7158809cf4f3c
  (this must match SHARED_KEY[] in BackscatterTag/Msp_FSK_secure.c)
  (demo is using the live receiver implementation under receiver/)

---- Scenario 1: Legitimate packet from the real tag ------------------------
  first packet, counter=100
    bytes:  000000649b6752f63025c05fe7f8c726
    -> ACCEPTED counter=100  reason=OK

---- Scenario 2: Replay attack (adversary records and re-emits) --------------
  same packet replayed
    bytes:  000000649b6752f63025c05fe7f8c726
    -> REJECTED counter=100  reason=replay: counter 100 <= last_accepted 100
  older recording (counter=50) replayed
    bytes:  000000323d6a184ff5826b1b11de27ac
    -> REJECTED counter=50  reason=replay: counter 50 <= last_accepted 100

---- Scenario 3: Tampering: single bit flip on a fresh-counter packet --------
    original:  000000c8eb26b8274602bc9ff22eb596
  after flipping one bit in the ciphertext
    bytes:  000000c8eb36b8274602bc9ff22eb596
    -> REJECTED counter=200  reason=MAC verification failed

---- Scenario 4: Forgery: adversary builds a packet without the real key -----
  packet built with all-zero key
    bytes:  0000012c33ffd82b7406a9bade4d7f75
    -> REJECTED counter=300  reason=MAC verification failed
  16 random bytes
    bytes:  9e4e9f0c6ca6ae6efb402597c7281a1b
    -> REJECTED counter=2655952652  reason=MAC verification failed

---- Scenario 5: Legitimate traffic still works after the attacks ------------
  new packet, counter=500
    bytes:  000001f4ba248c7acdfd66b25099f7d1
    -> ACCEPTED counter=500  reason=OK

======================================================================
Summary
======================================================================
  ACCEPTED: 2 legitimate packets (counters 100 and 500)
  REJECTED: 2 replays, 1 tamper, 2 forgeries

Without auth + replay protection, the on-air bytes 'AA 7E O P E N'
are publicly observable, so scenarios 2-4 would all OPEN THE DOOR.
PS C:\Users\andre\GitHub\Secure-Backscatter-IoT>
```

Figure 5. Output of the offline security demo script (demo_attacks.py).

5.5 Performance Summary

Metric	Measured / Observed Value
Excitation frequency	2.48 GHz (Pluto LO)
Sub-carrier frequencies	1.0 kHz ('1') / 1.7 kHz ('0')
Bit duration (wall-clock)	50 ms per chip
Packet length	18 bytes total (1 preamble + 1 sync + 4 counter + 4 ciphertext + 8 CMAC[0:8])

Metric	Measured / Observed Value
Packet airtime	≈ 7.2 s per packet at 50 ms/bit × 18 × 8 bits
Sideband SNR (typical)	+13 to +15 dB at ±1 kHz, with line-of-sight ≈ 1 m geometry
Tag duty cycle	Reed-switch gated; no transmission when magnet is absent

Compared to expectation, the link decodes reliably at sub-meter range with the current exciter and antenna setup, and the security layer correctly rejects all tested attack patterns. The dominant practical limitation is sideband SNR margin: at ranges beyond roughly one meter, dropouts increase noticeably. This is expected for a tag with no active RF amplification and a non-tuned wire antenna at 2.4 GHz, and is consistent with the path-loss budget for a single-antenna passive backscatter system at this scale.

6. Team Contribution

The contribution of each team member is summarized in Table 2.

Member	Contribution	Approx. share
Andrew Keefe	Exciter setup and Raspberry Pi service, receiver visualization (rx_monitor), security demo script, schematic and hardware documentation.	50%
Andrew Napieraj	Tag firmware (FSK + AES-CTR + CMAC + counter persistence), receiver DSP pipeline, system integration, debugging and testing.	50%

7. Conclusion

We built and demonstrated a battery-free authenticated backscatter communication link suitable for smart-lock and access-control applications. The tag uses no RF oscillator and consumes only the switching energy of an MSP430 GPIO and an ADG902 RF switch. Binary FSK on a 1.0 / 1.7 kHz sub-carrier, chosen specifically to avoid harmonic overlap, gives the receiver a robust differential decision per chip, eliminating the threshold-drift problems that limited an earlier OOK design we attempted. AES-CTR encryption, AES-CMAC authentication, and a monotonic counter persisted to flash provide cryptographic integrity, replay protection, and confidentiality, demonstrated through both live tag-to-receiver decodes and a dedicated offline attack-emulation script.

Key design features include: non-harmonic sub-carrier frequency selection to avoid receiver cross-contamination; multi-phase chip slicing for fading and boundary-transient robustness; a two-stage CFO tracking loop adapted from Satori; and reed-switch gating of the entire transmission path so the tag is RF-silent until physically actuated.

The main limitations are operating range and throughput. With our exciter power budget and antenna, the link is reliable up to roughly one meter; pushing further would require either an active LNA on the receive side, a properly tuned 2.4 GHz tag antenna, or both. Throughput is intentionally modest (one packet per several seconds) because we prioritized reliability over speed for a security-focused demo. Useful extensions include a hierarchical wake-up receiver on the tag side (along the lines of SyncScatter), a proper OFDM-compatible code-word

translation scheme to interoperate with commodity WiFi, and energy harvesting from the exciter signal so the tag has no battery at all.

8. References

[1] M. Dunna, M. Meng, P.-H. Wang, C. Zhang, P. Mercier, and D. Bharadia, "SyncScatter: Enabling WiFi-like Synchronization and Range for WiFi Backscatter Communication," Proc. 18th USENIX NSDI, 2021. https://www.usenix.org/conference/nsdi21/presentation/dunna

[2] P. Zhang, D. Bharadia, K. Joshi, and S. Katti, "HitchHike: Practical Backscatter Using Commodity WiFi," Proc. ACM SenSys, 2016.

[3] P. Zhang, C. Josephson, D. Bharadia, and S. Katti, "FreeRider: Backscatter Communication Using Commodity Radios," Proc. ACM CoNEXT, 2017.

[4] NIST Special Publication 800-38A, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," 2001.

[5] NIST Special Publication 800-38B, "Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication," 2005.

[6] J. Song, R. Poovendran, J. Lee, and T. Iwata, "The AES-CMAC Algorithm," RFC 4493, IETF, 2006. https://datatracker.ietf.org/doc/html/rfc4493

[7] Analog Devices, "ADG901/ADG902 Wideband 4 GHz, 40 dB Isolation at 1 GHz, CMOS 1.65 V to 2.75 V Absorptive/Reflective SPST Switches," Datasheet, Rev. C, 2013.

[8] Analog Devices, "EVAL-ADG901EBZ / EVAL-ADG902EBZ / EVAL-ADG918EBZ / EVAL-ADG919EBZ User Guide," UG-300.

[9] Texas Instruments, "MSP430G2x53, MSP430G2x13 Mixed Signal Microcontroller," SLAS735J Datasheet.

[10] Analog Devices, "ADALM-PLUTO Software-Defined Radio Active Learning Module," Product Documentation.

[11] A. Keefe, "Secure-Backscatter-IoT," GitHub repository, 2026. https://github.com/AndrewKeefe918/Secure-Backscatter-IoT

[12] "Satori: Practical Backscatter Receiver Synchronization," 2025. (See documents/2025.Satori.pdf in the project repository.)